Usuarios

Daniel Ayala, Inma Hernández, David Ruiz, Margarita Cruz, Carlos Arévalo

Contenido

1. Requisitos . . . 2
2. Requisitos de información (RI) . . . 2
   2.1. RI-1: Usuarios . . . 2
3. Reglas de negocio (RN) . . . 2
   3.1. RN-1: Mayor de edad . . . 2
   3.2. RN-2: Unicidad del correo electrónico . . . 2
4. Requisitos funcionales (RF) . . . 2
   4.1. RF-1: Informes simples de Usuarios . . . 2
   4.2. RF-2: Informes complejos de Usuarios . . . 2
5. Pruebas de aceptación (PA) . . . 3
   5.1. PA-1: Usuarios . . . 3
6. Modelo Conceptual . . . 3
   6.1. Diagrama de clases . . . 3
   6.2. Diagrama de objetos . . . 3
7. Modelo Relacional . . . 4
   7.1. Intensión . . . 4
   7.2. Extensión . . . 4
8. Álgebra relacional . . . 4
9. Pruebas HTTP . . . 5
10. Modelo Tecnológico (MariaDB) . . . 6
   10.1. Script SQL para crear la base de datos . . . 6
   10.2. Script SQL para la carga inicial de datos . . . 6
   10.3. Consultas . . . 7
   10.4. SQL Avanzado . . . 9
   10.5. Fecha de nacimiento en lugar de la edad . . . 9

Versión PDF disponible

1. Requisitos

Catálogo de Requisitos

2. Requisitos de información (RI)

2.1. RI-1: Usuarios

- Como: Profesor de la asignatura
- Quiero: Poder almacenar la siguiente información de los usuarios: nombre, edad, email y género (masculino, femenino, otro). Todos los atributos son obligatorios salvo el género.
- Para: Que el estudiante realice a partir de este requisito el modelo conceptual, relacional y tecnológico.

3. Reglas de negocio (RN)

3.1. RN-1: Mayor de edad

- Como: Profesor de la asignatura
- Quiero: Que los usuarios del sistema sean mayores de edad
- Para: Que el estudiante practique con restricciones simples.

3.2. RN-2: Unicidad del correo electrónico

- Como: Profesor de la asignatura
- Quiero: Que no existan dos usuarios en el sistema con el mismo correo electrónico
- Para: Que el estudiante practique con restricciones simples.

4. Requisitos funcionales (RF)

4.1. RF-1: Informes simples de Usuarios

- Como: Profesor de la asignatura
- Quiero: Que el sistema sea capaz de generar los siguientes informes:
  - Usuarios ordenados por nombre de la A a la Z
  - Nombre y correo de los usuarios de género femenino.
  - Nombre, edad y correo de los usuarios con dominio ‘@us.es’.
  - Edad media y total de usuarios.
  - Edad media y total de los usuarios con dominio ‘@us.es’.
- Para: Que el estudiante practique con consultas simples.

4.2. RF-2: Informes complejos de Usuarios

- Como: Profesor de la asignatura
- Quiero: Que el sistema sea capaz de generar los siguientes informes:
  - Edad media de los usuarios según el género.
  - Número de usuarios según el género.
  - Edad media de los usuarios según el género.
  - Total de usuarios según el género.
  - Usuarios de mayor edad.
  - Usuarios de mayor edad según el género.

  - Edad media y total de usuarios según el dominio de su correo electrónico.
- Para: Que el estudiante practique con consultas complejas.

5. Pruebas de aceptación (PA)

5.1. PA-1: Usuarios

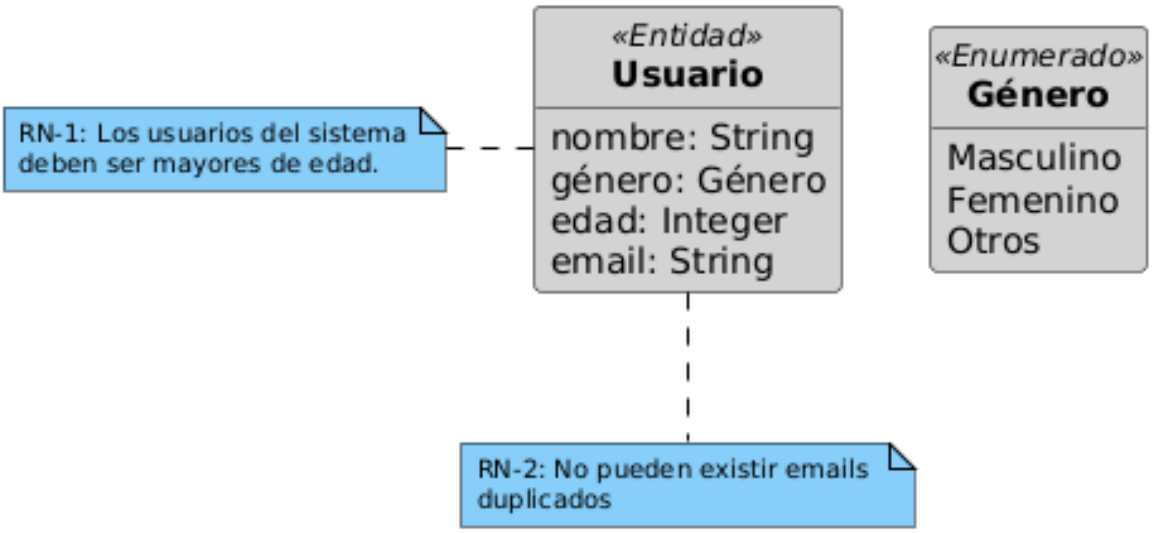
1. ✅ Crear un nuevo usuario con todos los datos correctos según las reglas de negocio
2. ✅ Crear un nuevo usuario sin especificar el género.
3. ❌ Crear un nuevo usuario sin nombre.
4. ❌ Crear un nuevo usuario sin edad.
5. ❌ Crear un nuevo usuario sin email.
6. ❌ Crear un nuevo usuario con el correo repetido.
7. ❌ Crear un nuevo usuario menor de edad.

6. Modelo Conceptual

6.1. Diagrama de clases

El modelo conceptual presenta la entidad Usuario como única clase del dominio, con atributos básicos de identificación y descripción personal.

- **Entidad principal**: Usuario con atributos nombre, edad, email y género (opcional).
- **Restricciones implícitas**: Email único para cada usuario; edad debe ser ≥ 18 años; género puede ser masculino, femenino u otro (o NULL).
- **Clave primaria**: Email actúa como identificador único natural.

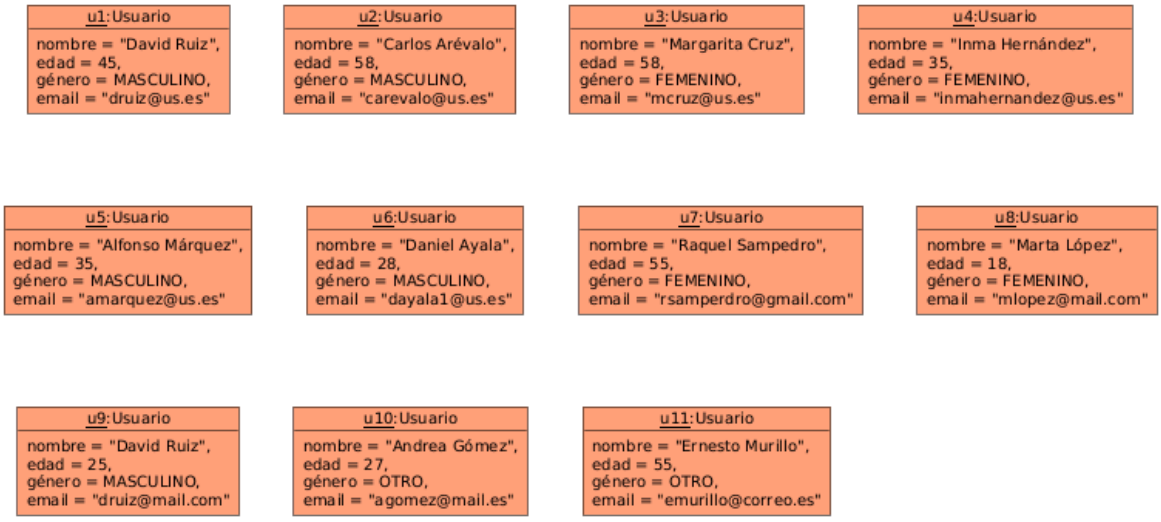


6.2. Diagrama de objetos

El diagrama de objetos ilustra instancias concretas de usuarios con diferentes combinaciones de valores en los atributos.

- **Ejemplos representativos**: Usuarios con géneros diversos (masculino, femenino, otro, ausente), edades variadas (todas ≥ 18), emails únicos.
- **Casos límite**: Usuario sin género especificado, usuarios con edades justas en el límite mínimo.

- **Variabilidad**: Los ejemplos muestran la diversidad permitida en nombres, dominios de email y opciones de género.



7. Modelo Relacional

7.1. Intensión

```
Usuarios(usuarioId, nombre, género, edad, email)
  PK(usuarioId)
  AK(email)
```

7.2. Extensión

```
Usuarios = {
  (u1,  "David Ruiz",      45, MASCULINO, "druiz@us.es"),
  (u2,  "Carlos Arévalo",  58, MASCULINO, "carevalo@us.es"),
  (u3,  "Margarita Cruz",  58, FEMENINO,  "mcruz@us.es"),
  (u4,  "Inma Hernández",  35, FEMENINO,  "inmahernandez@us.es"),
  (u5,  "Alfonso Márquez", 35, MASCULINO, "amarquez@us.es"),
  (u6,  "Daniel Ayala",     28, MASCULINO, "dayala1@us.es"),
  (u7,  "Raquel Sampedro", 55, FEMENINO,  "rsampedro@gmail.com"),
  (u8,  "Marta López",     18, FEMENINO,  "mlopez@mail.com"),
  (u9,  "David Ruiz",      25, MASCULINO, "druiz@mail.com"),
  (u10, "Andrea Gómez",     27, OTRO,      "agomez@mail.es"),
  (u11, "Ernesto Murillo",  55, OTRO,      "emurillo@correo.es")
}
```

8. Álgebra relacional

- Renombrado de la relación Usuarios (corregido para incluir el atributo género):

$$\underset{U(uid,n,g,ed,em)}{\rho}(Usuarios)$$

- Nombre y correo de las usuarias (género femenino):

$$Mujeres \leftarrow \underset{n,em}{\Pi}\left(\underset{g=\text{ FEMENINO}}{\sigma}(U)\right)$$

- Nombre, edad y correo de los usuarios con dominio “@us.es”:

$$UsuariosUS \leftarrow \underset{n,ed,em}{\Pi}\left(\underset{\text{dominio}(em)='us.es'}{\sigma}(U)\right)$$

Asumimos una función $\text{dominio}(\cdot)$ que dado un email devuelve su dominio (la cadena tras la @).

- Edad media y total de usuarios:

$$MedTotUsuarios \leftarrow \overset{\text{AVG}(ed),\ \text{COUNT}(*)}{\gamma}(U)$$

- Edad media y total de los usuarios con dominio "@us.es":

$$MedTotUsuariosUS \leftarrow \overset{\text{AVG}(ed),\ \text{COUNT}(*)}{\gamma}(UsuariosUS)$$

- Edad media de los usuarios agrupada por género:

$$MediaGenero \leftarrow \underset{g}{\overset{\text{AVG}(ed)}{\gamma}}(U)$$

- Número de usuarios agrupados por género:

$$TotalGenero \leftarrow \underset{g}{\overset{\text{COUNT}(*)}{\gamma}}(U)$$

- Edad media y total de usuarios según el dominio del correo electrónico:

$$MedTotDominio \leftarrow \underset{\text{dominio}(em)}{\overset{\text{AVG}(ed),\ \text{COUNT}(*)}{\gamma}}(U)$$

- Edad del usuario de mayor edad:

$$edadMayor \leftarrow \overset{\text{MAX}(ed)}{\gamma}(U)$$

- Edad máxima por género:

$$MayoresGenero \leftarrow \underset{g}{\overset{\text{MAX}(ed)}{\gamma}}(U)$$

9. Pruebas HTTP

Para llevar a cabo las pruebas HTTP usaremos silence, que nos facilitará la labor. Hay que tener en cuenta los siguientes aspectos:

- Configurar el proyecto para trabajar con la BD de Usuarios. En el archivo `settings.py` hay que proporcionar los datos de la conexión a la BD y los archivos SQL para crear la BD.
- Ejecutar `silence createdb`.
- Ejecutar `silence createapi`. Esto crea un archivo JSON en la carpeta `endpoints/auto` por cada tabla en la que se define el API Rest para acceder a la BD.
- En el primer cuatrimestre no usamos autenticación, por lo que hay que modificar el API creado automáticamente, haciendo una copia, para poner el atributo `auth_required` a false en todos los casos.
- Ejecutar `silence createtest`. Este comando crea un archivo `.http` en la carpeta `tests/auto` para cada tabla de la BD. Estos tests generados automáticamente sirven de guía para personalizar los tests.

A partir de las pruebas de aceptación definidas en el catálogo de requisitos las peticiones HTTP serían:

La respuesta que se obtendría para cada petición es la siguiente:

10. Modelo Tecnológico (MariaDB)

Para crear el esquema de la base de datos en MariaDB se puede usar el siguiente script:

10.1. Script SQL para crear la base de datos

Usuarios/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para crear la BD del ejercicio de Usuarios
--

-- Crear la base de datos
CREATE DATABASE IF NOT EXISTS UsuariosDB;
USE UsuariosDB;

DROP TABLE IF EXISTS Usuarios;

CREATE OR REPLACE TABLE Usuarios(
  usuarioId INT AUTO_INCREMENT NOT NULL,
  nombre VARCHAR(64) NOT NULL,
  genero ENUM('MASCULINO', 'FEMENINO', 'OTRO'),
  edad INTEGER NOT NULL,
  email VARCHAR(64) NOT NULL,
  PRIMARY KEY(usuarioId),
  CONSTRAINT RN_1_EdadAdulta CHECK (edad>=18), -- RN-1
  UNIQUE (email) -- RN-2
);
```

10.2. Script SQL para la carga inicial de datos

Para cargar los datos de prueba se puede usar el siguiente script:

Usuarios/populateDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para hacer la carga inicial del ejercicio de Usuarios
--


-- Primero dejamos la tabla de Usuarios vacía
DELETE FROM Usuarios;

-- Inserción de datos iniciales
INSERT INTO Usuarios(nombre, genero, edad, email)
VALUES
  ('David Ruiz', 'MASCULINO', 45, 'druiz@us.es'),
  ('Carlos Arévalo', 'MASCULINO', 58, 'carevalo@us.es'),
```

```
  ('Margarita Cruz', 'FEMENINO', 58, 'mcruz@us.es'),
  ('Inma Hernández', 'FEMENINO', 35, 'inmahernandez@us.es'),
  ('Alfonso Márquez', 'MASCULINO', 35, 'amarquez@us.es'),
  ('Daniel Ayala', 'MASCULINO', 28, 'dayala1@us.es'),
  ('Raquel Sampedro', 'FEMENINO', 55, 'rsampedro@gmail.com'),
  ('Marta López', 'FEMENINO', 18, 'mlopez@mail.com'),
  ('David Ruiz', 'MASCULINO', 25, 'druiz@mail.com'),
  ('Andrea Gómez', 'OTRO', 42, 'agomez@mail.es'),
  ('Ernesto Murillo', 'otro', 55, 'emurillo@correo.es');
```

10.3. Consultas

Para crear las consultas que implementan los requisitos funcionales se puede usar el siguiente script:

Usuarios/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Consultas para implementar los RFs del ejercicio de Usuarios
--


-- RF-1: Usuarios ordenados por nombre de la A-Z
SELECT *
  FROM Usuarios
  ORDER BY nombre ASC
;

-- RF-1: Nombre y correo de los usuarios de género femenino.
SELECT nombre, email
  FROM Usuarios
  WHERE genero='FEMENINO'
;

-- RF-1: Nombre, edad y correo de los usuarios con dominio 'us.es'.
SELECT nombre, edad, email
  FROM Usuarios
  WHERE email LIKE '%@us.es'
;

-- RF-1: Edad media y total de usuarios.
SELECT AVG(edad) mediaTotal, COUNT(*) total
  FROM Usuarios
;

-- RF-1: Edad media y total de los usuarios con dominio 'us.es'.
SELECT AVG(edad) mediaUS, COUNT(*) totalUS
  FROM Usuarios
  WHERE email LIKE '%@us.es'
;

-- RF-2: Edad media de los usuarios según el género.
SELECT genero, AVG(edad) media
  FROM Usuarios
  GROUP BY genero
```

```
;

-- RF-2: Número de usuarios según el género.
SELECT genero, COUNT(*) total
  FROM Usuarios
  GROUP BY genero
;

-- RF-2: Edad media de los usuarios según el género.
SELECT genero, AVG(edad) mediaEdad
  FROM Usuarios
  GROUP BY genero
;

-- RF-2: Total de usuarios según el género.
SELECT genero, COUNT(*) total
  FROM Usuarios
  GROUP BY genero
;

-- RF-2: Edad media y total de usuarios según el dominio de su correo electrónico.
SELECT AVG(edad), COUNT(*), SUBSTRING_INDEX(
    SUBSTR(email, INSTR(email, '@') + 1),' ',1) dominio
  FROM Usuarios
  GROUP BY dominio
; -- cálculo del dominio en SQL: https://stackoverflow.com/questions/2628138/how-
to-
  -- select-domain-name-from-email-address

-- RF-2: Usuarios de mayor edad
-- Opción 1: devuelve todos los usuarios con edad máxima
SELECT *
  FROM Usuarios
  WHERE edad = (
    SELECT MAX(edad) FROM Usuarios
  )
;

-- Opción 2: devuelve sólo un usuario
SELECT *
  FROM Usuarios
  ORDER BY edad DESC
  LIMIT 1;

-- RF-2: Usuario de mayor edad según el género
SELECT *
  FROM Usuarios u1
  WHERE edad = (
    SELECT MAX(u2.edad)
      FROM Usuarios u2
      WHERE u1.genero=u2.genero
  )
;
```

10.4. SQL Avanzado

Tenemos que implementar una función que calcula la edad de un usuario a partir de su fecha de nacimiento, y un trigger que comprueba que la fecha de nacimiento es anterior a la fecha actual y, además el usuario es mayor de edad.

El script para la función es el siguiente:

Usuarios/fGetAge.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Función para calcular la edad a partir de una fecha
--

DELIMITER //
  CREATE OR REPLACE FUNCTION fGetAge(birthDate DATE)
  RETURNS INT
  BEGIN
    DECLARE age INT;
    SET age = (SELECT TIMESTAMPDIFF(YEAR, birthDate, CURDATE()));
    RETURN age ;
  END //
DELIMITER ;
```

El script para el trigger es el siguiente:

Usuarios/tCheckAge.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script con triggers para Usuarios
--

-- Disparador para comprobar edad antes de insertar
DELIMITER //
CREATE OR REPLACE TRIGGER tCheckAge_RN_1
  BEFORE INSERT ON Usuarios
  FOR EACH ROW
  BEGIN
    IF ( NEW.edad < 18 ) THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RN-1: Los Usuarios tienen que
ser mayores de edad';
    END IF;
END//
DELIMITER ;
```

10.5. Fecha de nacimiento en lugar de la edad

En la creación de la tabla ahora hay que añadir que sustituir la edad por la fecha de nacimiento, pero ya no se puede comprobar que el Usuario es mayor de edad con un simple CHECK, y es necesario usar un TRIGGER, además para calcular la edad definiremos una función:

Ahora el script para la creación de la tabla es el siguiente:

Usuarios2/createDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para crear la BD del ejercicio de Usuarios2
--

-- Crear la base de datos
CREATE DATABASE IF NOT EXISTS UsuariosDB2;
USE UsuariosDB2;
-- Procidimiento para crear la tabla de Usuarios
DELIMITER //
CREATE OR REPLACE PROCEDURE createDB()
BEGIN
  DROP TABLE IF EXISTS Usuarios;
  CREATE OR REPLACE TABLE Usuarios(
    usuarioId INT AUTO_INCREMENT NOT NULL,
    nombre VARCHAR(64) NOT NULL,
    genero ENUM('MASCULINO', 'FEMENINO', 'OTRO'),
    fechaNacimiento DATE NOT NULL,
    email VARCHAR(64) NOT NULL,
    PRIMARY KEY(usuarioId),
    UNIQUE (email) -- RN-2
    -- Para comprobar que el Usuario es mayor de edad hace falta un Trigger
  );
END //
DELIMITER ;

-- Llamada para ejecutar el procedimiento
CALL createDB();
```

Los datos de prueba son los mismos que en el caso anterior pero sustituyendo la fecha de nacimiento por la edad:

Usuarios2/populateDB.sql

```sql
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script para hacer la carga inicial de Usuarios2
--

-- Procedimiento para hacer la carga inicial de la tabla de Usuarios
DELIMITER //
CREATE OR REPLACE PROCEDURE populateDB()
BEGIN
  -- Primero dejamos la tabla de Usuarios vacía
  DELETE FROM Usuarios;
  -- Inserción de datos iniciales
INSERT INTO Usuarios(nombre, genero, fechaNacimiento, email)
VALUES
  ('David Ruiz',      'MASCULINO', '1977-05-12', 'druiz@us.es'),
    ('Carlos Arévalo',  'MASCULINO', '1965-08-23', 'carevalo@us.es'),
    ('Margarita Cruz',  'FEMENINO',  '1968-02-10', 'mcruz@us.es'),
    ('Inma Hernández',  'FEMENINO',  '1985-11-30', 'inmahernandez@us.es'),
    ('Alfonso Márquez', 'MASCULINO', '1988-03-15', 'amarquez@us.es'),
```

```
    ('Daniel Ayala',    'MASCULINO', '1994-09-07', 'dayala1@us.es'),
    ('Raquel Sampedro', 'FEMENINO',  '1963-01-19', 'rsampedro@gmail.com'),
    ('Marta López',     'FEMENINO',  '2002-06-25', 'mlopez@mail.com'),
    ('Andrea Gómez',    'OTRO',      '1991-12-03', 'agomez@mail.es'),
    ('Ernesto Murillo', 'OTRO',      '1949-07-21', 'emurillo@correo.es')
;

END;
//
DELIMITER ;

-- Llamada al procedimiento
CALL populateDB();
```

Para las consultas podemos usar la función definida para calcular la edad o crear una vista de la tabla Usuarios que tendrá la edad siempre actualizada.

Usuarios2/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Consultas para implementar los RFs del ejercicio de Usuarios2
--

-- Tenemos dos opciones:
-- A.- Usar la función definida para calcular la edad
-- B.- Usar una vista de la tabla usuarios con la edad calculada

-- Ejemplo con la opción A
-- RF-1: Nombre, edad y correo de los usuarios con dominio 'us.es'.
SELECT nombre, fGetAge(fechaNacimiento) edad, email
  FROM Usuarios
  WHERE email LIKE '%@us.es'
;

-- Ejemplo con la opción B
CREATE OR REPLACE VIEW vUsuariosEdad AS
  SELECT *, fGetAge(fechaNacimiento) edad
    FROM Usuarios
;

SELECT nombre, edad, email
  FROM vUsuariosEdad
  WHERE email LIKE '%@us.es'
;

-- RF-2: Edad media y total de usuarios según el dominio de su correo electrónico.
SELECT AVG(edad), COUNT(*), fEmailDomain(email) dominio
  FROM vUsuariosEdad
  GROUP BY dominio
;
```

El trigger quedaría de la siguiente forma:

Usuarios2/tCheckAge.sql

```
--
-- Autor: David Ruiz
-- Fecha: Noviembre de 2022
-- Descripción: Script con triggers para Usuarios2
--

-- Disparador para comprobar edad antes de insertar

DELIMITER //
  CREATE OR REPLACE TRIGGER tCheckAge_RN_1
    BEFORE INSERT ON Usuarios
    FOR EACH ROW
    BEGIN
      IF ( NEW.fechaNacimiento > CURDATE() ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Fecha de nacimiento posterior
a la fecha actual';
      END IF;
      IF ( fGetAge(NEW.fechaNacimiento)<18 ) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'RN-1: Los Usuarios tienen que
ser mayores de edad';
      END IF;
  END//
DELIMITER ;
```